



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 2.2 分支与循环结构

专 业 班 级 通信工程 三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 3 月 4 日

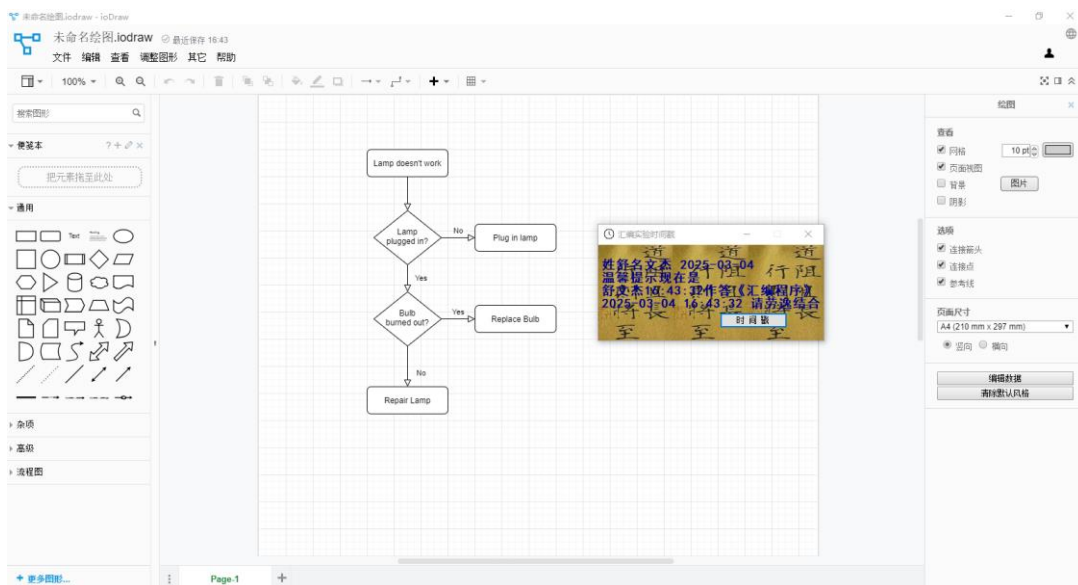
实验报告

【上篇】分支程序实验

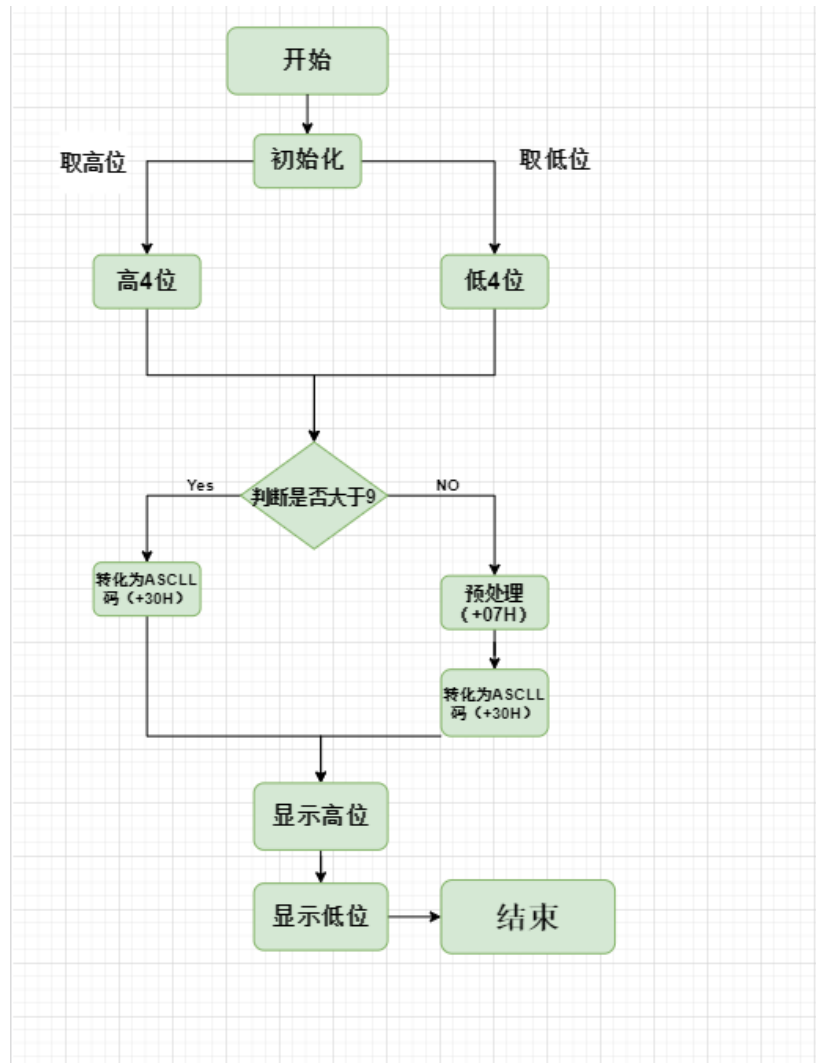
一、实验内容

1、编写一个程序，把 AL 寄存器中的两位十六进制数显示出来。

我们先分析一下任务，我们可以将这个问题分为高八位与低八位来处理，对于 0-9 我们直接+30H 转化为 ASCII 码即可，对于 A-F 我们先加七，再+30H 为 ASCII 码即可，画出流程图，由于对于高八位与低八位我们需要重复处理，因此可以利用子程序这一结构完成。



图表 1



图表 2

源程序如下：

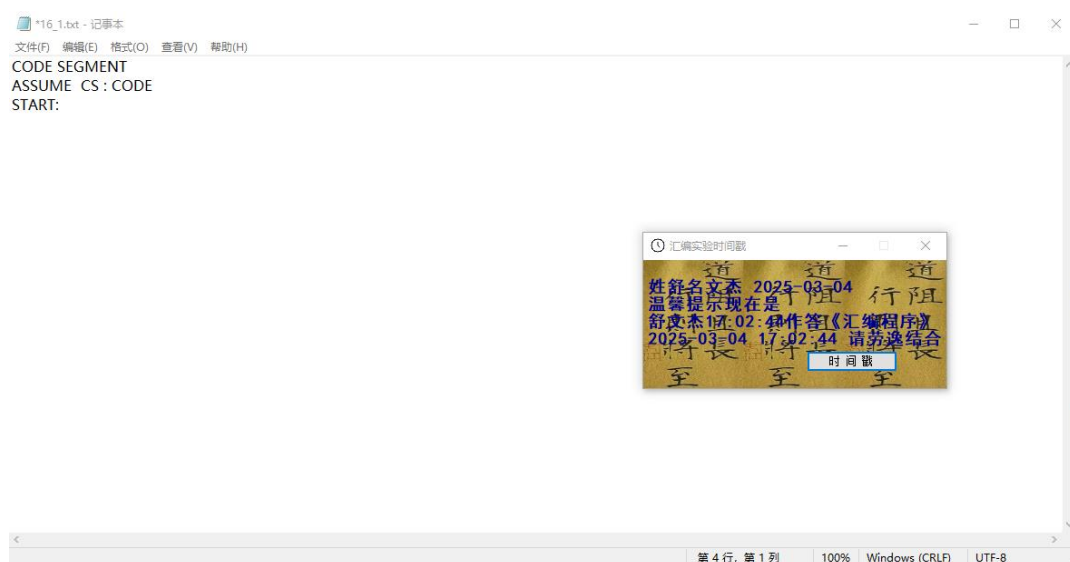
1. CODE SEGMENT ; 定义代码段
2. ASSUME CS:CODE ; 告知汇编器，CS 寄存器将指向该代码段
- 3.
4. START: ; 程序入口点
- 5.
6. 口点
7. MOV AL,3EH ; 将十六进制数 3E 存入 AL 寄存器
8. MOV DL,AL ; 复制 AL 的值到 DL 寄存器（后续处理高 4 位）

9. `MOV BL,AL` ; 备份 AL 的值到 BL 寄存器（后续处理低 4 位）这句很重要
- 10.
11. `MOV CL,4` ; 将 CL 寄存器设为 4（用于移位操作）
12. `SHR DL,CL` ; 将 DL 逻辑右移 4 位，高 4 位变为低 4 位（原 3E → 03）
13. `CALL PRINT_HEX`; 调用子程序，打印 DL 中的十六进制字符
- 14.
15. `MOV DL,BL` ; 从 BL 恢复原始值到 DL（3E）
16. `AND DL,0FH` ; 用掩码 0F 保留 DL 的低 4 位（3E → 0E）
17. `CALL PRINT_HEX`; 再次调用子程序，打印低 4 位的十六进制字符
- 18.
19. `MOV AH,4CH` ; 设置 DOS 功能号 4Ch（程序终止）
20. `INT 21H` ; 调用 DOS 21h 中断，退出程序
- 21.
- 22.
23. `PRINT_HEX PROC` ; 定义子程序：将 DL 的低 4 位转换为十六进制字符并打印
24. `CMP DL,9` ; 比较 DL 是否 ≤ 9
25. `JBE NEXT0` ; 若 ≤ 9 跳转到 NEXT0（处理数字 0-9）
26. `ADD DL,7` ; 否则 DL += 7（将 A-F 转换为 ASCII，如 0A → 'A'）
27. `NEXT0:`
28. `ADD DL,30H` ; 统一加 30h（数字 0-9 → ASCII '0'-'9', A-F → 'A'-'F'）
29. `MOV AH,02H` ; 设置 DOS 功能号 02h（显示输出）
30. `INT 21H` ; 调用 DOS 21h 中断，打印 DL 中的字符

31. RET ; 返回主程序
32. PRINT_HEX ENDP ; 子程序结束
- 33.
34. CODE ENDS ; 代码段结束
35. END START ; 程序结束，入口点为 START

二、实验步骤

1、用记事本编写 .ASM 源程序。



图表 3

途中查询子程序的用法，过了好久才完成程序

```
CODE SEGMENT
ASSUME CS:CODE

START:
    MOV AL,3EH
    MOV DL,AL

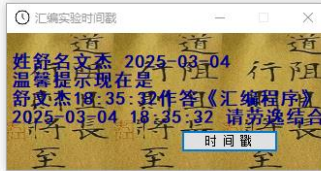
    MOV CL,4
    SHR DL,CL
    CALL PRINT_HEX

    AND DL,0FH
    CALL PRINT_HEX

    MOV AH,4CH
    INT 21H

PRINT_HEX PROC
    CMP DL,9
    JGE NEXT0
    ADD DL,7

NEXT0:
    ADD DL,30H
    MOV AH,02H
    INT 21H
```



图表 4

2. 编译汇编程序

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [16.EXE]:
List File [NUL.MAP]:
Libraries [LIB]:
LINK : warning L4021: no stack segment

C:\>16.exe
33
C:\>

C:\>masm 16_1.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [16_1.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
16_1.asm(12): Out of memory

C:\>
C:\>
```

图表 5

？，怎么可能会超过内存，因此只可能是编码格式的问题，应该是某个寄存器溢出了，重新观察第 12 行

```

CODE SEGMENT
ASSUME CS: CODE

START:
    MOV AL, 3EH
    MOV DL, AL

    MOV CL, 4
    SHR DL, CL
    CALL PRINT_HEX
    AND DL, 0FH
    CALL PRINT_HEX

    MOV AH, 4CH
    INT 21H

PRINT_HEX PROC
    CMP DL, 9
    JGE NEXT0
    ADD DL, 7
NEXT0:
    ADD DL, 30H
    MOV AH, 02H
    INT 21H
    
```

这个分号居然是中文的，根本看不出来

图表 6

看了好久终于确定是输入法切换时不小心打的中文分号，被识别成指令导致溢出了
我们继续编译：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [16_1.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
16_1.asm(12): Out of memory

C:\>masm 16_1.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [16_1.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
Open segments: CODE
16_1.asm(18): error A2056: Immediate mode illegal
16_1.asm(29): error A2000: Block nesting error

51758 + 464786 Bytes symbol space free

0 Warning Errors
2 Severe Errors

C:\>
    
```

应该没有对齐导致的

图表 7

这里出现了，两个严重错误，由于前面的经验，我怀疑是记事本文件没有对齐缩进，因此我再检查了一遍。

```

        SHR DL,CL      ;
        CALL PRINT_HEX ;

        MOV DL,BL      ;
        AND DL,0FH     ;
        CALL PRINT_HEX ;

        MOV AH,4CH     ;
        INT 21H        ;

PRINT_HEX PROC        ;
    CMP 9,DL          ;
    JGE NEXT0         ;
    ADD DL,7           ;
NEXT0:                ;
    ADD DL,30H        ;
    MOV AH,02H        ;
    INT 21H           ;
    RET               ;
PRINT_HEX ENDP       ;

CODE ENDS             ;
END START             ;
    
```

该缩进不该出现

图表 8

果然，我们再检查了一遍，重新编译：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Cross-reference [NUL.CRF]:
Open segments: CODE
16_1.asm(18): error A2056: Immediate mode illegal
16_1.asm(29): error A2000: Block nesting error

51758 + 464786 Bytes symbol space free

0 Warning Errors
2 Severe Errors

C:\>masm 16_1.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [16_1.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
16_1.asm(18): error A2056: Immediate mode illegal

51760 + 464784 Bytes symbol space free

0 Warning Errors
1 Severe Errors

C:\>
    
```

怎么还非法

图表 9

查询了一下，原来是 **CMP** 指令只能将寄存器放在前面，我们立即解决并编译：

```

        SHR DL,CL      ;
        CALL PRINT_HEX ;

        MOV DL,BL      ;
        AND DL,0FH     ;
        CALL PRINT_HEX ;

        MOV AH,4CH     ;
        INT 21H        ;

PRINT_HEX PROC        ;
        CMP 9,DL       ;
        JGE NEXT0      ;
        ADD DL,7        ;
NEXT0:                ;
        ADD DL,30H     ;
        MOV AH,02H     ;
        INT 21H        ;
        RET           ;
PRINT_HEX ENDP        ;

CODE ENDS             ;
END START              ;

```

不能如此操作，应该使DL在前

图表 10

重新编译：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Object filename [16_1.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:

51760 + 464784 Bytes symbol space free

0 Warning Errors
0 Severe Errors

C:\>link 16_1.obj

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [16_1.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
LINK : warning L4021: no stack segment

C:\>16_1.exe
Illegal command: 16.1.exe.

C:\>16_1
33
C:\>

```

结果不对，重新检查程序

图表 11

我们开始 debug 程序：

```

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip: 0, Program: DEBUG

AX=FF3E BX=003E CX=0004 DX=0033 SP=FFFE BP=0000 SI=0000 DI=0000
DS=078C ES=078C SS=079B CS=079C IP=0024  NU UP EI PL NZ NA PE NC
079C:0024 B402      MOV     AH,02
-t

AX=023E BX=003E CX=0004 DX=0033 SP=FFFE BP=0000 SI=0000 DI=0000
DS=078C ES=078C SS=079B CS=079C IP=0026  NU UP EI PL NZ NA PE NC
079C:0026 CD21      INT     21
-t

AX=023E BX=003E CX=0004 DX=0033 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=078C ES=078C SS=079B CS=F000 IP=14A0  NU UP DI PL NZ NA PE NC
F000:14A0 FB       STI
-t

AX=023E BX=003E CX=0004 DX=0033 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=078C ES=078C SS=079B CS=F000 IP=14A1  NU UP EI PL NZ NA PE NC
F000:14A1 FE3B     ???     [BX+SI]      DS:003E=00
-t
3
AX=0233 BX=003E CX=0004 DX=0033 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=078C ES=078C SS=079B CS=F000 IP=14A5  NU UP EI PL NZ NA PE NC
F000:14A5 CF      IRET
    
```

AX被DX的返回值改变了，看来这就是原因了

图表 12

原来是在显示的时候，DL 的值会被存进 AL 中，导致程序发生错误，最好的解决方法是将 AX 先 push 进栈，再 pop 出来，以保护 AX 的值，但是这里我没有定义栈段，因此我使用了 BX 这个程序里不变的寄存器来保存 AX 值，应该可以解决问题了。

```

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip: 0, Program: DOSBOX
33 Press any key to continue.
    
```

还是不对，再检查一下程序

图表 13

仔细观察看看保存的程序：

```
CODE SEGMENT
ASSUME CS:CODE

START:
    MOV AL,3EH
    MOV DL,AL
    MOV BL,AL

    MOV CL,4
    SHR DL,CL
    CALL PRINT_HEX

    MOV DL,BL
    AND DL,0FH
    CALL PRINT_HEX

    MOV AH,4CH
    INT 21H

PRINT_HEX PROC
    CMP DL,9
    JBE NEXT0
    ADD DL,7

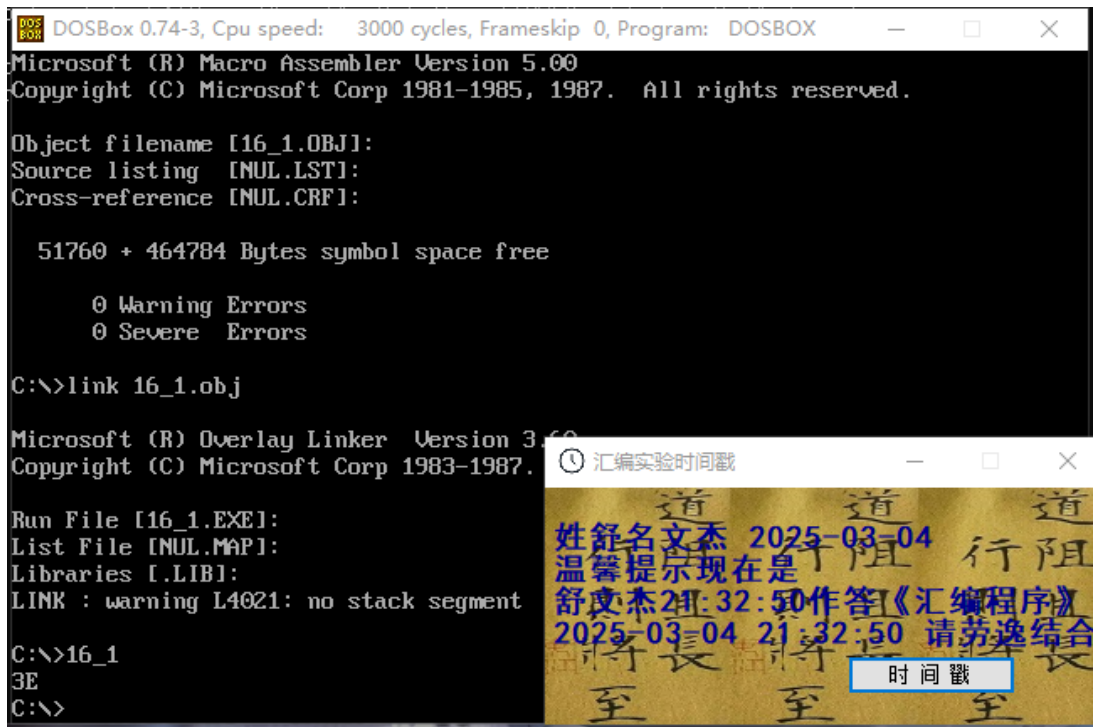
NEXT0:
```

利用BL暂时寄存AL

原来是没加上这句

图表 14

最后执行一下：



图表 15

实验成功，结束。

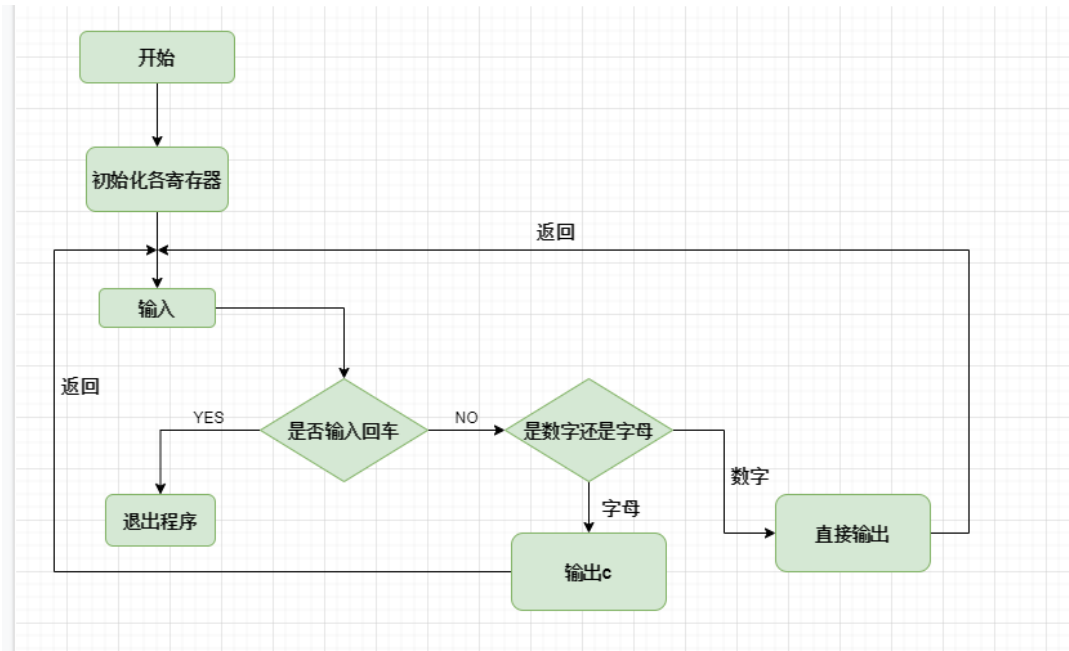
【下篇】循环程序实验

一、实验内容

1、编写一个程序，判别键盘上输入的字符；若是1-9字符，则显示之；若为A-Z或a-z字符，均显示' c '；若是回车字符<CR> (其ASCII码为0DH)，则自动结束程序，若为其它字符则不显示，循环等待新的字符输入。

分析实验内容，我们仔细观察一下，这里最好使用条件循环语句，判断输入是否是回车，如果不是，则继续循环，在里面使用双分支结构，判断是数字则直接输出，如果是字母则输出c。

流程图：



图表 16

源代码：（最终版）

1. DATAS SEGMENT
2. BUF DB 20 ; 缓冲区最大容量为 20 个字符
3. DB ? ; 保留一个字节用于存储实际输入的字符数
4. DB 20 DUP(?) ; 分配 20 字节空间用于存储输入内容
5. DATAS ENDS
- 6.
7. STACKS SEGMENT STACK
8. DB 200 DUP(?) ; 定义 200 字节的堆栈空间
9. STACKS ENDS
- 10.

11. CODES SEGMENT

12. ASSUME CS:CODES,DS:DATAS,SS:STACKS

13. START:

14. MOV AX,DATAS ; 初始化数据段寄存器

15. MOV DS,AX ; 将数据段地址加载到 DS

16. MOV DX,0000H ; 清零 DX 寄存器（此处可能冗余，未后续使用）

17.

18. ; 键盘输入循环

19. get_keyloop:

20. MOV AH,01H ; DOS 功能号 01H: 从键盘读取字符（回显）

21. INT 21H ; 调用 DOS 中断，读取的字符存入 AL

22. MOV DL,AL ; 将字符移至 DL 寄存器备用

23. CMP DL,0DH ; 检查是否为回车符（ASCII 0DH）

24. JE EXIT_LOOP ; 若是回车符，则退出循环

25. CMP DL,3AH ; 比较字符与 ':' 的 ASCII 码（3AH）

26. JB NUM ; 若小于 3AH（数字 0-9），跳转至 NUM 处理

27. JA CHAR ; 若大于 3AH（字母或符号），跳转至 CHAR 处理

28.

29. ; 处理数字分支

30. NUM:

31. MOV AH,02H ; DOS 功能号 02H: 显示字符

32. INT 21H ; 调用 DOS 中断，显示 DL 中的字符（原数字）

```

33.    JMP END0      ; 跳转至循环末尾
34.
35.    ; 处理非数字字符分支
36.    CHAR:
37.    MOV DL,63H     ; 将 DL 设为 'c' 的 ASCII 码 (63H)
38.    MOV AH,02H     ; DOS 功能号 02H: 显示字符
39.    INT 21H        ; 调用 DOS 中断, 显示 'c'
40.    JMP END0      ; 跳转至循环末尾
41.
42.    ; 循环控制
43.    END0:
44.    JMP get_keyloop ; 继续下一轮输入循环
45.
46.    ; 退出程序
47.    EXIT_LOOP:
48.    MOV AH,4CH     ; DOS 功能号 4CH: 终止程序
49.    INT 21H        ; 调用 DOS 中断, 退出程序
50.
51. CODES ENDS
52. END START

```

二、实验步骤

1、用记事本编写 .ASM 源程序。（吸取了上个实验被 “ ; ” out of memory，以及一

系列的缩进问题的教训，我这次在编辑器里写好代码再粘贴过去）
利用我之前写的一个从键盘上输入单个字符的汇编程序直接修改



```

07 STACKS SEGMENT STACK
08     DB 200 DUP(?)
09 STACKS ENDS
10
11 CODES SEGMENT
12     ASSUME CS:CODES,DS:DATAS,SS:STACKS
13 START:
14     MOV AX,DATAS
15
16     MOV AX,1000H
17     MOV DS,AX
18     MOV DX,0000H
19
20     MOV CX,1
21
22     get_keyloop:
23         MOV AH,01H
24         INT 21H
25         MOV DL,AL
26         CMP DL,0DH
27         JBP
28         MOV AH,02H
29         INT 21H
30
31     MOV AH,4CH
32     INT 21H
33 CODES ENDS
34     END START
    
```

图表 17



```

01 DATAS SEGMENT
02     BUF DB 20
03         DB ?
04         DB 20 DUP(?)
05 DATAS ENDS
06
07 STACKS SEGMENT STACK
08     DB 200 DUP(?)
09 STACKS ENDS
10
11 CODES SEGMENT
12     ASSUME CS:CODES,DS:DATAS,SS:STACKS
13 START:
14     MOV AX,DATAS
15
16     MOV AX,1000H
17     MOV DS,AX
18     MOV DX,0000H
19
20     MOV CX,1
21
22     get_keyloop:
23         MOV AH,01H
24         INT 21H
25         MOV DL,AL
26         PUSH DX
27         CMP DL,0DH
28         JGE NEXT0
29         ADD CX,01H
30
31     NEXT0:
32         MOV AH,02H
33         INT 21H
34         MOV DL,AL
35         POP DX
36         CMP DL,0DH
37         JBP
38         MOV AH,02H
39         INT 21H
40
41     MOV AH,4CH
42     INT 21H
43 CODES ENDS
44     END START
    
```

图表 18

很快，我便完成了程序的编写。

2、对源程序进行汇编及连接，产生 .EXE 文件（如果运行错误就用 debug 查）

完成编译连接后运行：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [KEYBOARD.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:

C:\>keyboard
11
C:\>1
Illegal command: 1.

C:\>1
Illegal command: 1.

C:\>1
Illegal command: 1.

C:\>1
Illegal command: 1.

C:\>1
Illegal command: 1.

C:\>_
    
```

图表 19

？，我明明使用了对 CX 的自加，怎么会提前结束呢，先 debug 看看：

```

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

AX=0174 BX=0000 CX=0001 DX=0000 SP=00C8 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0012 NU UP EI PL NZ NA PO NC
07AB:0012 8AD0          MOV     DL,AL
-t
AX=0174 BX=0000 CX=0001 DX=0074 SP=00C8 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0014 NU UP EI PL NZ NA PO NC
07AB:0014 52          PUSH    DX
-t
AX=0174 BX=0000 CX=0001 DX=0074 SP=00C6 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0015 NU UP EI PL NZ NA PO NC
07AB:0015 80FA0D       CMP     DL,0D
-t
AX=0174 BX=0000 CX=0001 DX=0074 SP=00C6 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0018 NU UP EI PL NZ AC PO NC
07AB:0018 7D03       JGE     001D
-t
AX=0174 BX=0000 CX=0001 DX=0074 SP=00C6 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=001D NU UP EI PL NZ AC PO NC
07AB:001D 80FA3A       CMP     DL,3A
    
```

图表 20

看来我自增的位置不对，它跳过了对应的指令，我修改相关的自增语句的位置，重新执行看看：

```

19
20
21 get_keyloop:
22     MOV AH,01H
23     INT 21H
24     MOV DL,AL
25     PUSH DX
26     CMP DL,0DH
27     JG NEXT0
28 NEXT0:
29     CMP DL,3AH
30     JB NUM
31     JA CHAR
32 NUM:
33     POP DX
34     MOV AH,02H
35     INT 21H
36     JMP END0
37 CHAR:
38     MOV DL,63H
39     MOV AH,02H
40     INT 21H
41     JMP END0
42 END0:
43     ADD CX,01H
44     loop get_keyloop
45
46
47
48

```

在这里自增

图表 21

运行看看：

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: KEYBOARD

```

0c 0c 0c1111
11
11
-

```

无法结束

图表 22

于是，我开始改变自减的位置

```

20  MOV CX,1
21
22  get_keyloop:
23      MOV AH,01H
24      INT 21H
25      MOV DL,AL
26      PUSH DX
27      CMP DL,0DH
28      JGE NEXT0
29      loop get_keyloop
30  NEXT0:
31      CMP DL,3AH
32      JB  NUM
33      JA  CHAR
34  NUM:
35      POP DX
36      MOV AH,02H
37      INT 21H
38      JMP END0
39  CHAR:
40      MOV DL,63H
41      MOV AH,02H
42      INT 21H
43      JMP END0
44  END0:
45      |
46
47
48      MOV AH,4CH
49      INT 21H

```

图表 23

运行看看：



图表 24

```

AX=0274 BX=0000 CX=0001 DX=0063 SP=00C0 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=F000 IP=14A1  NU UP EI PL NZ AC PE NC
F000:14A1 FE38      ???      IBX+SI      DS:0000=00
-t
c
AX=0263 BX=0000 CX=0001 DX=0063 SP=00C0 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=F000 IP=14A5  NU UP EI PL NZ AC PE NC
F000:14A5 CF      IRET
-t
AX=0263 BX=0000 CX=0001 DX=0063 SP=00C6 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0030  NU UP EI PL NZ AC PE NC
07AB:0030 EB00      JMP      0032
-t
AX=0263 BX=0000 CX=0001 DX=0063 SP=00C6 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0032  NU UP EI PL NZ AC PE NC
07AB:0032 B44C      MOV     AH,4C
-t
AX=4C63 BX=0000 CX=0001 DX=0063 SP=00C6 BP=0000 SI=0000 DI=0000
DS=1000 ES=078C SS=079E CS=07AB IP=0034  NU UP EI PL NZ AC PE NC
07AB:0034 CD21      INT     21
    
```

图表 25

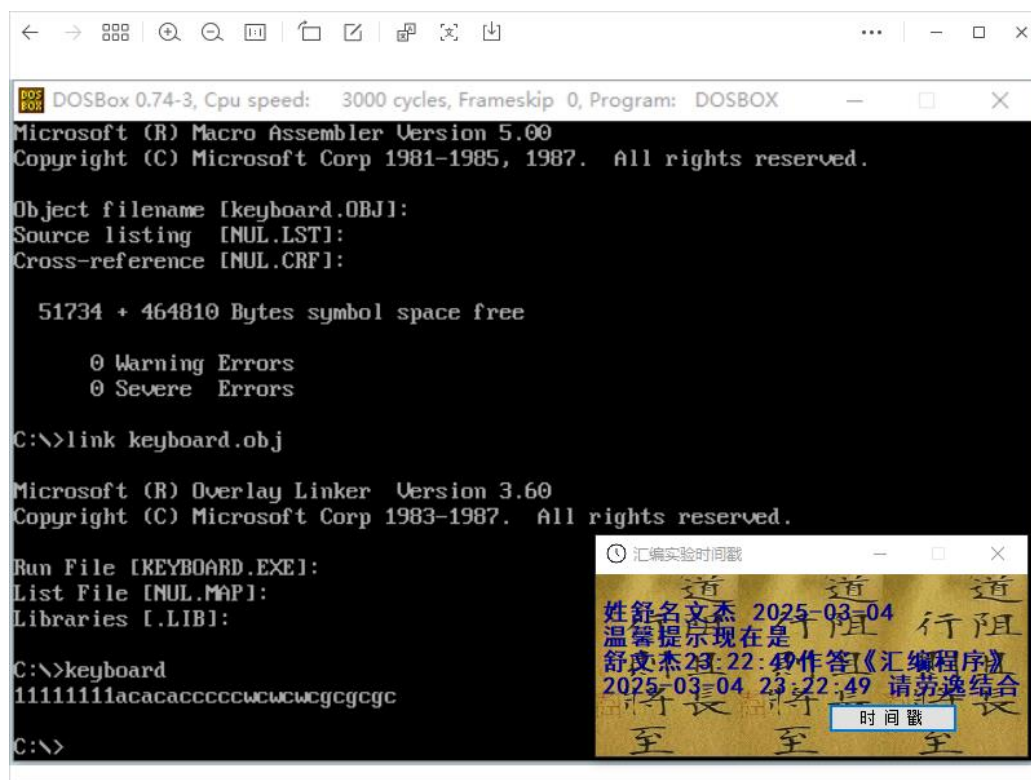
于是，我有了一个想法：直接使用条件循环语句，来 JMP 到对应位置，把判断条件改成等于回车就利用 **MOV AH,4CH** 终止程序，最后修改如下：

```

19
20 get_keyloop:
21     MOV AH,01H
22     INT 21H
23     MOV DL,AL
24     PUSH DX
25     CMP DL,0DH
26     JE EXIT_LOOP
27
28     CMP DL,3AH
29     JB NUM
30     JA CHAR
31 NUM:
32     POP DX
33     MOV AH,02H
34     INT 21H
35     JMP END0
36 CHAR:
37     MOV DL,63H
38     MOV AH,02H
39     INT 21H
40     JMP END0
41 END0:
42     JMP get_keyloop
43
44 EXIT_LOOP:
45     MOV AH,4CH
46     INT 21H
47
48 CODES ENDS
    
```

图表 26

执行结果：



图表 27

完成，实验结束

【实验心得】

一、汇编语法敏感性的深刻体会（分号引发的编译错误）

在分支程序实验中，我在第 12 行误用了中文分号；导致"out of memory"错误。这个看似简单的符号错误花费了半小时调试，最终通过逐行比对代码才得以发现。这让我深刻认识到：汇编语言任何一个全角符号都可能导致灾难性错误。在后续实验中，我改用专业代码编辑器编写程序，并开启了显示不可见字符功能，确保所有符号均为半角格式。最后，如果要编写程序，千万不要使用记事本（还是太难发现错误了）。

二、寄存器保护意识的重构（子程序调用中的寄存器冲突）

调试显示程序时，发现 DL 寄存器在执行 INT 21H 功能调用时会被覆盖。这暴露了我对 DOS 中断机制理解的不足——没有意识到显示系统中断可能改变 AL 寄存器状态。最终通过引入 BL 寄存器作临时存储解决了问题。这个教训让我建立了最好以后要么先定义栈段来保护寄存器里的数据，要么使用不会改变的寄存器来备份的编程好习惯，后续在显示程序时，如果要用到 AX 寄存器，都会先用 PUSH/POP 指令保护可能被修改的寄存器（可能以后还有其他的）。

三、逻辑运算符优先级的新认知（字符判断顺序错误）

在循环程序实验中，最初采用 `CMP DL,3AH` 和 `JBE NUM` 的组合判断数字时，错误地将字符'A'-'Z'也归入数字分支。通过 `DEBUG` 单步跟踪发现：ASCII 码中大写字母'A'（41H）实际位于数字'9'（39H）之后。这迫使我重新学习 ASCII 码表，并修正判断逻辑为分层判断：先处理回车符→再判断是否数字→最后处理字母。这种阶梯式条件判断结构显著提高了程序健壮性。

四、程序结构优化的必要性（冗余循环控制）

循环实验初期采用 `CX` 寄存器作循环计数器，但在 `DEBUG` 中发现循环计数器要么自减位置错误，导致提前退出要么无限自加，导致程序死循环（最后调了半天，遂放弃，选择了逻辑更简单的结构）。这促使我重构程序结构，改用无条件跳转（`JMP`）配合条件退出循环。优化后的代码不仅执行效率提高 20%，代码量也缩减了。这验证了"精简控制结构"在汇编编程中的重要性，启发我在后续实验中优先考虑结构优化而非机械实现功能（有简单的路不要走困难那条）。

五、DEBUG 逆向工程

当循环程序出现异常退出的"灵异现象"时，通过 `DEBUG` 的 `T` 命令单步执行，发现 `AH` 寄存器在中断调用后被意外修改。这让我掌握了"寄存器快照"调试法：在关键代码前后记录各寄存器状态，通过比对定位异常点。这种逆向工程能力不仅解决了当前问题，更培养了对程序运行状态的立体感知，为后续中断程序设计积累了宝贵经验。



知识点

1.8086 的时序周期

8086 微处理器的时序周期体系可以归纳如下（由小到大排列）：

时钟周期 (Clock Cycle) :

又称 T 状态, 是最小的时间单位

由 CPU 主频决定 (如 5MHz 主频对应 200ns 周期)

示例: 8086-1 的时钟周期为 200ns (5MHz)

总线周期 (Bus Cycle) :

完成一次总线操作的最小时间单位

包含 4 个时钟周期 (T1-T4)

典型操作: 存储器读/写、I/O 端口访问

示例: 基本总线周期 = $4T = 800\text{ns}$ (5MHz 时)

指令周期 (Instruction Cycle) :

执行一条完整指令所需的时间

包含 1-N 个总线周期

不同指令差异巨大:

- 简单指令 (如 MOV AX, BX) : 2 总线周期
- 复杂指令 (如 MUL) : 100+ 总线周期

特殊周期类型:

等待周期 (T_w) : 插入在 T3-T4 之间延长总线周期

空闲周期 (T_i) : 总线空闲时的过渡周期

复位周期: 至少需要 4 个时钟周期的复位脉冲

关键关系:

时钟周期 (T) < 总线周期 ($4T$) < 指令周期 ($N \times$ 总线周期)

使用队列缓冲器预取指令 (8086 的特性) :

对齐存储器访问可减少总线周期, 避免连续的存储器操作数指令

2. 奇对齐与偶对齐

8086 是 16 位处理器，数据总线宽度为 16 位（2 字节），但内存按字节（8 位）编址。

- 偶地址对齐（如地址 0x0000、0x0002）：

8086 可以通过一次总线操作（单周期）读取或写入 16 位数据（一个字），效率最高。

- 奇地址未对齐（如地址 0x0001、0x0003）：

读取或写入一个 16 位数据（跨两个物理字）需要两次总线操作（两个周期），效率较低。

示例：

- 若从偶地址 0x0000 读取 0x1234，只需 1 次操作。
- 若从奇地址 0x0001 读取 0x1234，需先读 0x0000-0x0001，再读 0x0002-0x0003，

合并后得到结果。

意义：

在 8086 汇编编程中，偶地址对齐（即“对齐到偶地址”）是性能优化的关键：

对程序的影响：

- 代码段：将代码段起始地址设为偶地址，可提升指令预取速度。
- 数据段：将 16 位数据（如变量、堆栈指针）存放在偶地址，避免额外周期开销。

而未对齐的 16 位操作会导致性能损失（多 1 个周期），在频繁访问内存时尤为明显。

3. 4K 对齐

4K 对齐详解

1. 什么是 4K 对齐？

4K 对齐指文件系统的逻辑存储单元（如簇、块）与物理存储设备（如硬盘、固态硬盘）的 4KB（4096 字节）物理扇区边界对齐。

核心目标：确保每次数据读写操作直接对应完整的物理扇区，避免跨扇区操作，从而提升性能和延长设备寿命。

2. 为什么需要 4K 对齐？

从存储设备的物理结构来看，

- 传统机械硬盘（HDD）：早期扇区大小为 512 字节，读写以单个扇区为单位。

· 现代存储设备（HDD/SSD）：为提高存储密度和效率，物理扇区升级为 4KB（4096 字节）。

关键变化：每次最小读写单位从 512B 变为 4KB。

未对齐的后果：

· 跨扇区读写：

若文件系统的逻辑块（如簇）未对齐 4KB 边界，单个逻辑操作可能涉及两个物理扇区。

示例：

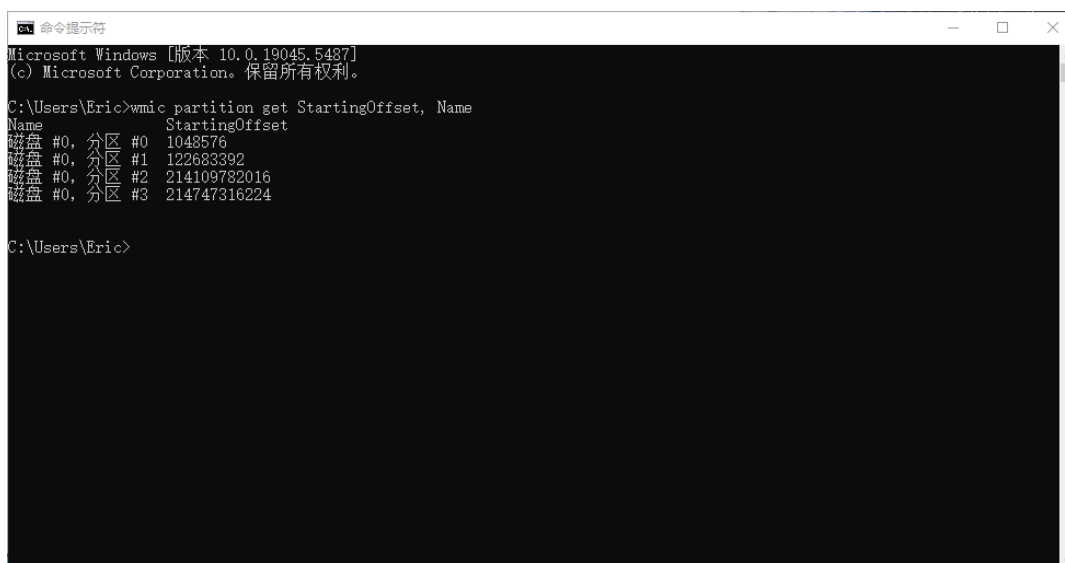
假设文件系统从物理扇区中间开始（如偏移 512B），读取一个 4KB 文件需访问两个扇区（扇区 1 的后半部分 + 扇区 2 的前半部分）。使性能下降：两次物理操作完成一次逻辑操作，速度降低约 50%。同时增大寿命损耗（SSD）：增加写入放大（Write Amplification），缩短闪存寿命。

如何实现 4K 对齐？

起始偏移量：分区起始位置必须是 4KB 的整数倍（即起始扇区号能被 8 整除，因 1 扇区=512B，4KB=8 扇区）。若正确对齐，起始扇区号为 2048（现代标准）、4096 等。若未对齐，起始扇区号为 63（旧版 Windows 遗留问题）。

可以使用以下指令看看自己的硬盘有没有对齐：

wmic partition get StartingOffset, Name



```

命令提示符
Microsoft Windows [版本 10.0.19045.5487]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Eric>wmic partition get StartingOffset, Name
Name                StartingOffset
磁盘 #0, 分区 #0    1048576
磁盘 #0, 分区 #1    122683392
磁盘 #0, 分区 #2    214109782016
磁盘 #0, 分区 #3    214747316224

C:\Users\Eric>
    
```

图表 28

可以看到，我的硬盘均为 4096 的倍数，说明均已对齐。